

Distributed Transactions: 2PC, 3PC, Saga

2PC (Two-Phase Commit)

- **What it is:** Classic distributed transaction protocol.\
 - **How:**
 1. **Prepare phase** -- coordinator asks participants if they can commit.\
 2. **Commit phase** -- if all say "yes," coordinator tells them to commit; else rollback.\
 - **Guarantees:** Strong consistency (all-or-nothing).\
 - **Cons:** Blocking (if coordinator dies after prepare but before commit, participants are stuck).\
 - **Use case:** Banking (money transfer between accounts in different DBs), critical systems where atomicity is mandatory.
-

3PC (Three-Phase Commit)

- **What it is:** Extension of 2PC with an extra "**pre-commit**" phase to reduce blocking.\
 - **How:**
 1. **CanCommit** -- ask if participants *can* commit.\
 2. **PreCommit** -- coordinator tells participants to prepare & acknowledge.\
 3. **DoCommit** -- final commit.\
 - **Guarantees:** Non-blocking under some failure scenarios.\
 - **Cons:** More network overhead, still not 100% safe under partition (CAP says nope). Rarely used in real systems.\
 - **Use case:** Research/academic, or ultra-high reliability distributed databases (not common in practice).
-

Saga

- **What it is:** Long-lived distributed transaction broken into a **sequence of local transactions**, each with a **compensating action** if it fails.\
 - **Types:**
 - **Choreography:** each service publishes events; other services listen & react.\
 - **Orchestration:** central coordinator tells each service what to do.\
 - **Guarantees:** Eventual consistency (not atomic), but highly available & scalable.\
 - **Cons:** Complex compensation logic, debugging harder.\
 - **Use case:** Microservices (e-commerce order → payment → inventory → shipping).
-

TL;DR

- **2PC** → Strong consistency, blocking, critical financial ops.\
- **3PC** → Adds safety vs. coordinator failure, but rarely used.\
- **Saga** → Eventual consistency, best for microservices & long workflows.